

1 chapter2 归纳法

1.1 归纳算法举例

1.1.1 多项式求值

假设有 $n + 2$ 个实数 $a_0, a_1, \dots, a_n$ 和 x 的序列，现在要对以下多项式求值：

$$P_n(x) = \sum_{i=0}^n a_i x^i$$

我们发现：

$$p_n(x) = (((((a_n x + a_{n-1})x + a_{n-2})x + a_{n-3})\dots))x + a_0$$

所以，我们得到具体表达式为：

$$p_{n-1}(x) = a_n x^{n-1} + a_{n-1} x^{n-2} + \dots + a_2 x + a_1$$

所以，有递推关系：

$$p_j(x) = x p_{j-1}(x) + a_{n-j}$$

Input: $n + 2$ 个实数 $a_0, a_1, \dots, a_n$ 和 x 的序列

Output: 表达式 $P_n(x)$ 的值

```
1  $p \leftarrow a_n$ 
2 for  $j \leftarrow 1$  to  $n$  do
3   |  $p \leftarrow xp + a_{n-j}$ 
4 end
5 return  $p$ 
```

Algorithm 1: HORNER

1.1.2 生成排列

显然，对于重复次数 $f(n)$ 有以下递推关系：

$$f(n) = n f(n-1) + n, n \geq 2$$

令 $f(n) = n! h(n)$, 那么

$$n! h(n) = n(n-1)! h(n-1) + n$$

即

$$h(n) = h(n-1) + \frac{1}{(n-1)!}$$

解得

$$h(n) = h(1) + \sum_{i=1}^{n-1} \frac{1}{i!} < \sum_{i=1}^{\infty} \frac{1}{i!} = e - 1$$

所以

$$f(n) = n! h(n) < (e-1)n! < 2 \cdot n!$$

Input: 正整数 n

Output: 所有长度为 n 的排列

```
1 for  $j \leftarrow 1$  to  $n$  do
2    $P[j] \leftarrow j$ 
3 end
4 perm1(1)
5 perm1(m)
6 if  $m = n$  then
7   return P // output P
8 else
9   for  $j \leftarrow m$  to  $n$  do
10    swap( $P[j], P[m]$ )
11    perm1( $m+1$ )
12    swap( $P[j], P[m]$ ) //  $P[m..n] = m, m+1, \dots, n$ 
13  end
14 end
```

Algorithm 2: PERMUTATIONS1

而输出语句的复杂度是 $\Theta(n)$ ，所以最终 $T(n) = \Theta(n \cdot n!)$

这里用到的思想其实是一种累加法的变体。也就是考虑递推式

$$f(n) = g(n)f(n-1) + h(n)$$

只需进行合适的换元，令 $p(n)$ 满足：

$$f(n) = \left(\prod_{i=1}^n g(i)\right) \cdot p(n)$$

这样得到：

$$\left(\prod_{i=1}^n g(i)\right) \cdot p(n) = \left(\prod_{i=1}^n g(i)\right) \cdot p(n-1) + h(n)$$

$$p(n) = p(n-1) + \frac{1}{\prod_{i=1}^n g(i)} h(n)$$

$$p(n) = p(1) + \sum_{i=1}^n \frac{1}{\prod_{j=1}^i g(j)} h(i)$$

由此，即可求出 $f(n)$ ，不过要注意无穷级数的作用。

1.1.3 基数排序

令 $L = \{a_1, a_2, \dots, a_n\}$ 是一张有 n 个数的集合，其中每个数恰好有 k 位数字，就是说每个数具有 $d_k d_{k-1} \dots d_1$ 形式，这里 d_i 是 0 到 9 中的一个数字。

我们研究一种排序算法对 L 进行非降序排列，使算法在几乎所有的实际应用中都以线性时间运行。

在这个问题中，我们对整数的大小 k 施行归纳，而不是对数 n 这个对象来用归纳法。

时间复杂度： $\Theta(kn)$ ，空间复杂度： $\Theta(10n)$

Input: 一个有 n 个数的表 $L = \{a_1, \dots, a_n\}$ 和 k 位数字

Output: 按非非降序排列的表 L

```
1 for  $j = 1$  to  $k$  do
2   准备 10 个空表  $L_0, L_1, \dots, L_9$  while  $L \neq \emptyset$  do
3      $a \leftarrow L.top()$ 
4      $L.pop()$ 
5      $i \leftarrow a$  的第  $j$  位数字, 将  $a$  加入表  $L_i$  中。// 取个位为第 1 位数字
6   end
7    $L \leftarrow L_0$ 
8   for  $i = 1$  to 9 do
9      $L \leftarrow L \cup L_i$ 
10  end
11 end
12 return  $L$ 
```

Algorithm 3: 基数排序

1.1.4 寻找多数元素

令 $A[1..n]$ 是一个整数序列, A 中的整数 a 如果在 A 中出现的次数多于 $\lfloor n/2 \rfloor$, 那么 a 称为多数元素。

- 初始, 令候选元素 $c = A[1]$, 并将计数器置 1
- 从 $A[2]$ 开始, 逐个扫描元素, 如果被扫描的元素和 c 相等, 则计数器加 1; 如果元素不等于 c , 则计数器减 1
- 如果所有的元素都已经扫描完毕并且计数器大于 0, 那么返回 c 作为多数元素的候选者
- 如果在 c 和 $A[j]$ ($1 < j < n$) 比较时计数器为 0, 那么对于 $A[j+1..n]$ 上的元素递归调用 candidate 过程。

时间复杂度 $\Theta(n)$, 空间复杂度 $\Theta(1)$

1.1.5 棋子移动游戏

洛谷题: <https://www.luogu.com.cn/problem/P1259>

对于最后处理的 4 对棋子是需要特殊对待的, 不然会在 2 白 2 空 2 黑的时候卡死。

1.2 本章习题

1.2.1 RADIXSORT 算法时间复杂性

当输入由下列区间中的 n 个正整数组成时, 以 n 为大小说明算法 RADIXSORT 的时间复杂性。

1. $[1 \dots n]$

Input: 一个数组 A 和整数 m, n

Output: 数组 A 中的一个候选者

```
1  $j \leftarrow m; c \leftarrow A[m]; count \leftarrow 1;$ 
2 while  $j < n$  and  $count > 0$  do
3    $j \leftarrow j + 1;$ 
4   if  $A[j] = c$  then
5      $count \leftarrow count + 1;$ 
6   else
7      $count \leftarrow count - 1;$ 
8   end
9 end
10 if  $j = n$  then
11   return  $c$  // 获取候选者
12 else
13   return  $candidate(j + 1)$ 
14 end
```

Algorithm 4: candidate(m)

2. $[1 \dots n^2]$

3. $[1 \dots 2^n]$

1.2.2 寻找名人问题

在一个房间里有 n 个人，其中一个名人。所谓名人，就是大家都认识他，但是他却不认识任何其他人。名人之外的其他人可能认识房间里另外的一部分人。你可以问房间里任何人问题，但是问题只能是：“你认识某某吗？”，对方回答“Yes”或者“No”。请设计一个时间复杂度为 $\Theta(n)$ 的归纳算法，通过问问题，把房间中的名人找出来。

1. 初始化候选人：假设第一个人为候选名人 $candidate = 0$ 。
2. 遍历排除非名人：
3. 从第 2 个人到第 n 个人，依次与候选人比较：
 - 若候选人认识当前人，则候选人不是名人，更新候选人为当前人。
 - 若候选人不认识当前人，则当前人不是名人，保留候选人。
4. 验证候选人合法性：
 - 检查是否所有人认识候选人。
 - 检查候选人是否不认识任何人。

Input: n 个元素的数组 $A[1..n]$

Output: 若存在则输出多数元素，否则输出 none

```
1  $c \leftarrow \text{candidate}(1)$ ;  
2 // 获取候选者  
3  $count \leftarrow 0$ ;  
4 for  $j \leftarrow 1$  to  $n$  do  
5   if  $A[j] = c$  then  
6      $count \leftarrow count + 1$ ;  
7     // 候选者进行计数  
8   end  
9 end  
10 if  $count > \lfloor n/2 \rfloor$  then  
11   return  $c$ ;  
12 end  
13 else  
14   return none;  
15 end
```

Algorithm 5: MAJORITY

表 1: 基数排序 (RADIXSORT) 的时间复杂性分析

输入区间	时间复杂度	解释
$[1 \dots n]$	$O(n)$	最大位数 $d = \lceil \log_{10} n \rceil \approx O(1)$ ，视为常数，故线性时间。
$[1 \dots n^2]$	$O(n^2)$	位数 $d = \lceil 2 \log_{10} n \rceil$ ，但数字范围扩大，近似为 $d = O(n)$ 。
$[1 \dots 2^n]$	$O(n \cdot 2^n)$	位数 $d = \lceil n \log_{10} 2 \rceil = \Theta(n)$ ，且输入规模为 2^n 量级。